

SERVICIO NACIONAL DE APRENDIZAJE

Centro Minero — Sogamoso, Boyacá

MANUAL TÉCNICO

Sistema de Gestión de Maquinaria Pesada

Versión 2.0

Campo	Valor
Versión del sistema	2.0
Framework principal	Django 5.2.6
Base de datos	MySQL 8.0 / SQLite (desarrollo)
Zona horaria	America/Bogota
Idioma de interfaz	Español (es-es)
Fecha de elaboración	23 de abril de 2026
Clasificación	Documento Técnico Interno

1. Introducción

1.1 Propósito del Documento

El presente Manual Técnico describe de manera exhaustiva la arquitectura, los módulos funcionales, los modelos de datos, los flujos de trabajo y los mecanismos de seguridad del Sistema de Gestión de Maquinaria Pesada SENA v2.0. El documento está dirigido al equipo de ingeniería de software responsable del desarrollo, mantenimiento, auditoría y escalamiento de la plataforma.

1.2 Alcance

El sistema comprende ocho aplicaciones Django independientes que cubren la gestión integral del ciclo de vida de la maquinaria pesada: registro y monitoreo de máquinas, control de inventario de componentes, generación de reportes, gestión documental, administración del sistema, visión artificial para detección de maquinaria y una API REST para integración con sistemas externos.

1.3 Audiencia

- Ingeniero/a líder de proyecto — revisión de arquitectura y decisiones de diseño.
- Desarrolladores backend/frontend — implementación y mantenimiento de módulos.
- Equipo de QA — referencia de flujos y validaciones esperadas.
- Administrador de infraestructura — despliegue, base de datos y backups.

1.4 Convenciones Tipográficas

Convención	Significado
`código`	Nombre de clase, método, campo o parámetro técnico
MAYÚSCULAS	Constante de configuración o evento del sistema
→	Flujo de ejecución o dependencia directa entre componentes
FK	Foreign Key (clave foránea)
M2M	ManyToManyField (relación muchos-a-muchos)

2. Stack Tecnológico y Configuración Global

2.1 Dependencias Python

La siguiente tabla relaciona cada dependencia del archivo requirements.txt con su responsabilidad técnica dentro del sistema:

Paquete	Versión / Ref.	Responsabilidad técnica
Django	5.2.6	Framework web: ORM, middleware, autenticación, admin, templates
djangorestframework	latest	API REST: ViewSets, serializers, paginación, autenticación por token
django-cors-headers	latest	Middleware CORS para peticiones cross-origin (frontend / apps móviles)
django-environ	latest	Gestión de variables de entorno desde archivo .env
mysqlclient	latest	Conector nativo CPython entre el ORM de Django y MySQL 8.0
Pillow	latest	Procesamiento de imágenes: ImageField, redimensionamiento, conversión
openpyxl	latest	Lectura y escritura de archivos Excel .xlsx
reportlab	latest	Generación programática de PDFs (fichas técnicas, reportes)
PyMuPDF	latest	Extracción de texto, TOC y metadatos de archivos PDF
python-docx	latest	Lectura de archivos Word .docx: párrafos, headings, tablas
opencv-python	latest	Captura de cámara web, decodificación y preprocesamiento BGR
numpy	latest	Operaciones vectoriales sobre embeddings faciales de 128 dimensiones
deepface	latest	Extracción de embeddings faciales (modelo Facenet, backend OpenCV)

Paquete	Versión / Ref.	Responsabilidad técnica
tf-keras	latest	Backend TensorFlow/Keras requerido por DeepFace para inferencia
celery	latest	Ejecución de tareas asíncronas: reportes, indexación en segundo plano
redis	latest	Message broker para Celery + caché de sesiones y rate limiting
apscheduler	latest	Programación de tareas periódicas (backup automático cada 2 horas)
qrcode[pil]	latest	Generación de códigos QR en PNG para identificación de maquinaria

2.2 Configuración de Base de Datos

El sistema utiliza MySQL 8.0 como motor de base de datos en entornos de producción y SQLite para desarrollo local. La configuración en settings.py define un único alias default con charset utf8mb4 para soporte completo de Unicode (incluyendo caracteres especiales del español y emojis en campos de texto):

Motor: django.db.backends.mysql | BD: proyecto_base | Host: localhost | Puerto: 3306 | Charset: utf8mb4

2.3 Parámetros Globales Relevantes

Parámetro	Valor configurado	Observación
DEBUG	True	Debe establecerse en False en producción
ALLOWED_HOSTS	['*']	Restringir a dominios específicos en producción
LANGUAGE_CODE	'es-es'	Internacionalización en español de España
TIME_ZONE	'America/Bogota'	UTC-5, sin horario de verano
SESSION_COOKIE_AGE	86400 s (24 h)	Duración de sesión autenticada

Parámetro	Valor configurado	Observación
EMAIL_HOST	smtp.gmail.com:587 TLS	Envío de correos de recuperación y bienvenida
VISION_CONFIDENCE_THRESHOLD	0.5	Umbral mínimo de confianza para detección visual
PAGE_SIZE (DRF)	50 registros	Paginación por defecto de la API REST

2.4 Middleware — Orden de Ejecución

El orden del middleware determina la cadena de procesamiento de cada solicitud HTTP. Los componentes se ejecutan de forma secuencial en la solicitud entrante y en orden inverso en la respuesta saliente:

- CorsMiddleware — gestiona cabeceras CORS antes del procesamiento de seguridad
- SecurityMiddleware — aplica HTTPS redirect, HSTS y cabeceras de seguridad
- SessionMiddleware — habilita el sistema de sesiones basado en cookies
- CommonMiddleware — normalización de URLs y cabeceras comunes
- CsrfViewMiddleware — validación de tokens CSRF en métodos POST/PUT/PATCH/DELETE
- AuthenticationMiddleware — asocia el usuario autenticado al objeto request
- MessageMiddleware — sistema de mensajes flash entre peticiones
- XFrameOptionsMiddleware — prevención de clickjacking mediante cabecera X-Frame-Options

3. Arquitectura General del Sistema

3.1 Patrón Arquitectónico

El sistema implementa el patrón MTV (Model-Template-View) de Django, que corresponde funcionalmente al patrón MVC. Cada aplicación Django encapsula su propia capa de modelos, vistas y plantillas, lo que garantiza alta cohesión y bajo acoplamiento entre módulos. La capa de persistencia es gestionada exclusivamente por el ORM de Django, sin consultas SQL directas en código de aplicación.

3.2 Estructura de Aplicaciones (Apps)

App Django	Responsabilidad principal
usuarios	Autenticación (contraseña + facial), gestión de roles, sesiones y recuperación de credenciales
maquinaria	Ciclo de vida completo de máquinas: registro, estados, alertas, mantenimiento y archivo histórico
inventario	Gestión de piezas y componentes con control de vida útil y captura fotográfica multi-ángulo
reportes	Generación asíncrona de reportes en PDF, Excel, CSV y JSON; métricas de rendimiento
documentos	Repositorio documental con indexación full-text, control de acceso granular y extracción automática
sistema	Configuración global, backups automáticos, logs de auditoría, notificaciones y centros de formación
vision	Detección de maquinaria mediante modelo TensorFlow Lite con pipeline de preprocesamiento OpenCV
api	API REST con Django REST Framework: CRUD, endpoints especializados y autenticación por token

3.3 Estructura de Directorios

El proyecto sigue la convención estándar de Django con separación explícita entre código de aplicación, archivos estáticos, archivos de medios generados por usuarios y scripts de utilidad:

- proyecto_base/app_prototipo/ — Configuración central Django (settings, urls, wsgi, asgi)
- proyecto_base/<app>/ — Código de cada aplicación: models.py, views.py, urls.py, serializers.py

- media/ — Archivos subidos: fotos de máquinas, piezas, usuarios, documentos, reportes y backups
- static/ — Archivos estáticos fuente: CSS, JS, Bootstrap 5.3 e Icons 1.10 (todos locales, sin CDN)
- staticfiles/ — Salida de collectstatic para servicio en producción
- scripts/ — Scripts de población de BD y verificación de integridad

3.4 Capa de Infraestructura

Componente	Tecnología	Función
Base de datos principal	MySQL 8.0	Persistencia de todos los modelos de aplicación
Caché / Message broker	Redis	Broker de Celery + caché de sesiones y rate limiting
Cola de tareas	Celery	Procesamiento asíncrono de reportes e indexación
Almacenamiento de archivos	Sistema de archivos local /media/	Imágenes, PDFs, backups, reportes generados
Programador de tareas	APScheduler	Backups automáticos cada 2 horas
Modelo de visión	TensorFlow Lite (.tflite)	Inferencia de detección de maquinaria sin GPU
Servidor SMTP	Gmail SMTP (TLS 587)	Envío de correos de sistema

4. Módulo de Usuarios

La aplicación usuarios centraliza la gestión de identidad, autenticación y control de acceso de todos los actores del sistema. Implementa un modelo de usuario personalizado desacoplado del `AbstractUser` de Django, lo que permite extender los campos sin modificar el esquema de autenticación nativo.

4.1 Modelo de Dominio

TipoUsuario

Define los roles del sistema mediante un campo permisos de tipo `JSONField` que almacena un mapa granular de accesos por módulo. Los roles predeterminados incluyen Administrador, Instructor, Aprendiz e Invitado, cada uno con diferentes alcances de permiso.

Usuario (modelo principal)

Campo clave	Tipo	Restricción / Comportamiento
numero_documento	CharField	UNIQUE — validado por regex de solo dígitos
email	EmailField	UNIQUE — identificador de autenticación secundario
estado	CharField (choices)	activo / inactivo / suspendido / pendiente — default: pendiente
tipo_usuario	FK → TipoUsuario	ON DELETE PROTECT — no se puede eliminar un rol en uso
centro_formacion	FK → CentroFormacion	ON DELETE SET_NULL
ficha	FK → Ficha	ON DELETE SET_NULL — exclusivo para rol Aprendiz
ultimo_acceso	DateTimeField(null)	Actualizado en cada inicio de sesión exitoso

ReconocimientoFacial

Relación `OneToOne` con `Usuario`. Almacena el embedding vectorial de 128 dimensiones generado por el modelo `Facenet` de `DeepFace`. Nunca se persisten imágenes de rostros; únicamente el vector numérico resultante del proceso de extracción.

4.2 Backend de Autenticación

La clase `UsuarioBackend` hereda de `ModelBackend` y sobrescribe el método `authenticate` para permitir login mediante `numero_documento` o `email` (a diferencia del comportamiento nativo que solo acepta `username`). Además, sincroniza los flags `is_staff` e `is_active` del `DjangoUser` interno basándose en el nombre del `TipoUsuario` y el estado del `Usuario` personalizado respectivamente.

4.3 Servicio de Reconocimiento Facial

Parámetro del servicio	Valor	Justificación técnica
Modelo de embedding	Facenet (128D)	Balance óptimo entre precisión y tiempo de inferencia
Backend de detección	OpenCV	Sin dependencias externas de GPU; compatible con entorno SENA
Métrica de distancia	Similitud coseno	Invariante a escala; robusta ante variaciones de iluminación
Umbral de autenticación	≥ 0.6	Configurable; equilibrio entre FAR y FRR
Rate limiting	5 intentos / 5 min / IP	Mitigación de ataques de fuerza bruta sobre endpoint facial

4.4 Flujo de Autenticación

Login por contraseña

- El cliente envía `numero_documento` (o `email`) y `password` vía POST a `/usuarios/login/`.
- `UsuarioBackend.authenticate()` localiza el `Usuario` y sincroniza el `DjangoUser` interno.
- Se valida el hash PBKDF2-SHA256 (260.000 iteraciones) con `check_password()`.
- Tras autenticación exitosa, se crea un registro `SesionUsuario` con token único, IP y `user-agent`.
- Se actualiza el campo `ultimo_acceso` y se redirige al dashboard.

Login por reconocimiento facial

- El cliente envía `numero_documento` e `imagen_base64` vía POST a `/usuarios/api/login-facial/`.
- Se verifica el rate limit por IP (máximo 5 intentos en 5 minutos con `django.core.cache`).

- FacialRecognitionService decodifica la imagen base64 a np.ndarray BGR y valida calidad mínima.
- DeepFace extrae el embedding vectorial de 128 dimensiones usando el modelo Facenet.
- Se calcula la similitud coseno entre el embedding capturado y el almacenado en ReconocimientoFacial.
- Si similitud ≥ 0.6 , se inicia sesión y se registra IntentoReconocimientoFacial(resultado='exitoso').

5. Módulo de Maquinaria

Es el módulo central del sistema. Gestiona el ciclo de vida completo de cada máquina pesada, desde su registro inicial hasta su eliminación con preservación histórica. Expone más de 60 rutas URL y su modelo Maquina actúa como ancla de referencia para los módulos de inventario, documentos, reportes y visión.

5.1 Modelos

Modelo	Propósito
Maquina	Entidad central: datos técnicos, estado operacional, código QR, responsable y auditoría
CategoriaMaquina	Clasificación tipológica con icono Bootstrap e identificador de color hexadecimal
Proveedor	Datos comerciales del proveedor de adquisición con calificación de 1 a 5
AlertaMaquina	Registro de incidencias con tipo, prioridad (baja / media / alta / critica) y estado de resolución
HistorialMaquina	Log inmutable de todos los eventos del ciclo de vida de cada máquina
MantenimientoProgramado	Planificación y ejecución de mantenimientos con desglose de costos y personal
MaquinaEliminada	Archivo JSON completo de una máquina eliminada; garantiza trazabilidad permanente
UsoMaquinaria	Registro de sesiones de uso en prácticas de formación por ficha e instructor
ObjetoMaquinaria	Componentes individuales con 6 fotografías angulares para reconocimiento visual

5.2 Ciclo de Vida de una Máquina

Registro

La vista crear_maquina_view valida la unicidad de codigo_inventario y numero_serie, instancia el objeto Maquina con created_by asignado al usuario autenticado y genera automáticamente un evento HistorialMaquina de tipo creacion.

Modificación

editar_maquina_view compara campo a campo el estado previo con los datos del formulario, registra únicamente los campos modificados en un nuevo HistorialMaquina de tipo actualizacion, incluyendo el valor anterior y el nuevo para auditoría completa.

Cambio de estado

La vista cambiar_estado_maquina ejecuta una transacción que actualiza el campo estado y registra un evento HistorialMaquina('cambio_estado') con el valor previo y el nuevo valor, retornando una respuesta JSON para consumo por AJAX.

Eliminación con backup

Antes de ejecutar maquina.delete(), el sistema recopila todos los datos relacionados (historial, alertas, mantenimientos) y los serializa en un registro MaquinaEliminada. Los campos datos_completos_maquina, historial_completo, alertas_asociadas y mantenimientos_realizados garantizan la trazabilidad permanente incluso después de la eliminación física del registro.

5.3 Sistema de Códigos QR

La vista generar_qr_maquina construye una URL absoluta hacia el endpoint info_qr_maquina/<codigo>/ y genera una imagen PNG del código QR usando la librería qrcode (PIL, 10 px/caja, borde 4). La imagen se retorna directamente como HttpResponse con content_type='image/png', sin persistencia en disco. El endpoint de información es accesible sin autenticación para permitir su uso mediante dispositivos móviles en campo.

5.4 Mantenimiento Programado

El modelo MantenimientoProgramado permite la planificación detallada con asignación de técnico principal, personal adicional (M2M), listas de componentes a revisar, herramientas y repuestos necesarios (JSONField). El desglose de costos incluye costo_estimado, costo_real, costo_repuestos y costo_mano_obra. El método marcar_completado() actualiza el estado, registra el evento en HistorialMaquina y recalcula la fecha de proximo_mantenimiento en la entidad Maquina.

6. Módulo de Inventario

Gestiona el inventario de piezas, repuestos y componentes asociados a la maquinaria. El modelo `PiezaInventario` implementa un control de vida útil basado en horas de uso, con alertas automáticas cuando una pieza alcanza su límite operacional o se deteriora.

6.1 Modelo `PiezaInventario` — Campos Destacados

Campo	Tipo	Descripción técnica
<code>condicion</code>	<code>CharField (choices)</code>	nueva / usada / cambiada / dañada
<code>horas_uso / horas_uso_maximas</code>	<code>DecimalField</code>	Control cuantitativo de vida útil
<code>maquina</code>	<code>FK → Maquina (SET_NULL)</code>	Asociación opcional a máquina específica
<code>foto_frontal ... foto_inferior</code>	<code>6 x ImageField</code>	Capturas angulares desde cámara web (base64 → archivo)
<code>tiene_alerta (@property)</code>	<code>bool</code>	True si <code>condicion='dañada'</code> O <code>horas_uso >= horas_uso_maximas</code>

6.2 Captura Multi-ángulo

La vista `nueva_pieza_view` recibe seis imágenes codificadas en base64 desde el cliente (capturadas mediante la API `MediaDevices` del navegador) y las convierte a objetos `ImageField`. Esto permite el reconocimiento visual posterior desde cualquier ángulo mediante el módulo de visión artificial.

7. Módulo de Reportes

Proporciona capacidad de generación bajo demanda de reportes en múltiples formatos, con soporte para procesamiento asíncrono mediante Celery cuando el volumen de datos lo requiere.

7.1 Formatos de Salida

Formato	Librería	Caso de uso típico
PDF	ReportLab	Fichas técnicas, informes ejecutivos con tablas y estilos corporativos
Excel (.xlsx)	OpenPyXL	Datos tabulares para análisis posterior en hojas de cálculo
CSV	csv (stdlib)	Integración con sistemas externos o importación en bases de datos
JSON	json (stdlib)	Consumo directo por APIs o aplicaciones frontend

7.2 Flujo de Generación

- El cliente realiza POST a /reportes/generar/ con tipo de reporte, filtros y formato deseado.
- Se crea un registro Reporte (PK: UUID) con estado='pendiente'.
- Si Celery está disponible, se encola la tarea asíncronamente y el estado pasa a 'generando'.
- El generador consulta la BD con los filtros especificados, construye el archivo y lo persiste en FileField.
- El estado final es 'completado'; el cliente descarga mediante GET /reportes/<uuid>/descargar/.
- Cada descarga incrementa el contador veces_descargado y actualiza fecha_ultima_descarga.

7.3 Métricas de Rendimiento

El modelo MetricasRendimiento almacena indicadores calculados automáticamente por período (diario, semanal, mensual, anual) segmentados por centro de formación y categoría de máquina. Incluye disponibilidad operacional, eficiencia promedio, tasas de mantenimiento y desglose de costos para análisis temporal del desempeño del parque de maquinaria.

8. Módulo de Documentos

Implementa un repositorio centralizado de documentación técnica con extracción automática de contenido, indexación full-text y control de acceso granular por nivel de confidencialidad.

8.1 Niveles de Acceso

Nivel	Audiencia autorizada
publico	Cualquier usuario autenticado en el sistema
interno	Usuarios con estado 'activo' en cualquier rol
confidencial	Exclusivo para roles Administrador y Coordinador
restringido	Solo los usuarios listados en el campo M2M usuarios_acceso del documento

8.2 Extracción Automática de Contenido

Al momento de subir un documento, el sistema detecta el tipo de archivo y ejecuta el extractor correspondiente. Para archivos PDF se utiliza PyMuPDF (fitz), extrayendo el texto completo página por página, el índice estructurado mediante doc.get_toc() y los metadatos del documento. Para archivos DOCX se emplea python-docx, procesando párrafos, headings por nivel de estilo y contenido de tablas. El resultado se almacena en contenido_texto (para búsqueda full-text) e indices_búsqueda (JSON con headings y palabras clave). Adicionalmente se calcula el checksum SHA-256 del archivo para verificación de integridad.

9. Módulo de Sistema

Proporciona las funciones de administración transversal: configuración dinámica, backups de base de datos, auditoría de actividad, notificaciones y gestión de la estructura organizacional de centros de formación.

9.1 Configuración Dinámica

El modelo Configuración permite modificar parámetros del sistema en tiempo de ejecución sin necesidad de reiniciar el servidor. Cada registro almacena una clave única, el valor como cadena de texto y un tipo_valor que indica cómo debe deserializarse (string, integer, float, boolean, json, date, datetime, email, url). El método get_valor_typed() realiza la conversión automática al tipo nativo de Python correspondiente.

9.2 Sistema de Backups Automáticos

APScheduler ejecuta una tarea programada cada 2 horas que genera un volcado completo de la base de datos mediante mysqldump (o equivalente SQLite), lo comprime en formato .sql.gz y lo almacena en /media/backups/%Y/%m/. El proceso crea y actualiza un registro BackupDatabase con estado, tamaño en bytes, número de tablas y registros procesados, y versiones de Django y Python vigentes. La restauración es posible desde la interfaz web para registros con estado 'completado' o 'restaurado' que tengan el archivo físico disponible.

9.3 Log de Auditoría

LogActividad registra cada acción significativa del sistema con granularidad de módulo (usuarios, maquinaria, reportes, documentos, sistema, backups, api), nivel de severidad (debug, info, warning, error, critical), usuario ejecutor, objeto afectado (tipo y PK), método y ruta HTTP, y tiempo de ejecución de la vista. Los índices en (timestamp), (usuario, timestamp) y (modulo, timestamp) garantizan consultas eficientes en la interfaz de auditoría.

9.4 Sistema de Notificaciones

El modelo Notificación (PK: UUID) permite el envío de alertas contextuales a usuarios específicos con tipología (info, success, warning, error, system), opción de acción requerida con URL y texto de botón, y vinculación opcional a una máquina o alerta específica. Los índices en (usuario, estado) garantizan consultas rápidas de notificaciones pendientes por usuario.

10. Módulo de Visión Artificial

Implementa detección automática de maquinaria pesada mediante un modelo TensorFlow Lite optimizado para inferencia en CPU, sin requerir aceleración por GPU. El pipeline de preprocesamiento utiliza OpenCV para garantizar compatibilidad con imágenes provenientes de diferentes fuentes.

10.1 Pipeline de Detección

- Verificación de disponibilidad del modelo .tflite. Si no está disponible, retorna {modelo_disponible: False}.
- Decodificación de bytes de imagen a np.ndarray en espacio de color BGR mediante OpenCV.
- Conversión BGR → RGB y redimensionamiento a 224×224 píxeles (dimensión de entrada del modelo).
- Normalización de valores de píxel al rango [0, 1] como tensor float32.
- Inferencia mediante TFLite Interpreter (interpreter.invoke()).
- Extracción de output_data, ordenamiento por confianza descendente.
- Aplicación del umbral VISION_CONFIDENCE_THRESHOLD (0.5); si no se supera, detectado=False.
- Mapeo de la clase detectada a su CategoriaMaquina correspondiente en el sistema.

10.2 Clases Reconocidas

Tipo de elemento	Clases del modelo
Maquinaria completa — Excavación	excavadora, bulldozer, retroexcavadora
Maquinaria completa — Elevación	grua_torre, grua_movil
Maquinaria completa — Nivelación	compactadora, motoniveladora, pavimentadora
Maquinaria completa — Transporte	cargador_frontal, camion_minero, volquete
Maquinaria completa — Perforación	perforadora
Componentes (no máquinas)	cucharon, orugas, brazo_hidraulico, cabina_operador, motor_diesel, entre otros

11. API REST

La capa de API REST, implementada con Django REST Framework, expone los datos del sistema para integración con clientes externos, aplicaciones móviles o sistemas de terceros. Toda la comunicación requiere autenticación por token (TokenAuthentication) o por sesión (SessionAuthentication).

11.1 Endpoints Principales

Método	Ruta	Descripción
POST	/api/auth/token/	Obtener token de acceso con credenciales
GET	/api/maquinas/	Listado paginado de máquinas (filtros: categoría, estado, centro)
GET	/api/maquinas/<id>/historial/	Historial completo de eventos de una máquina
POST	/api/maquinas/<id>/cambiar-estado/	Transición de estado con registro en historial
GET	/api/alertas/activas/	Alertas con estado 'activa' ordenadas por prioridad
POST	/api/alertas/<id>/resolver/	Resolución de alerta con notas y usuario ejecutor
POST	/api/reportes/generar/	Solicitud de generación asíncrona de reporte
GET	/api/reportes/<uuid>/descargar/	Descarga del archivo generado con control de acceso
GET	/api/dashboard/datos/	Datos agregados para renderizado de dashboard
GET	/api/estadisticas/generales/	Estadísticas globales del sistema
POST	/api/bulk/importar-maquinas/	Importación masiva desde archivo Excel o CSV

11.2 Formato de Respuesta de Error

Todos los errores de la API retornan un objeto JSON estructurado con tres campos: error (descripción legible), detail (información técnica adicional) y code (identificador de error para consumo programático). Esto garantiza consistencia en el manejo de excepciones por parte de los clientes de la API.

12. Seguridad y Validación

12.1 Hashing de Contraseñas

El sistema utiliza PBKDF2 con SHA-256 y 260.000 iteraciones (valor por defecto de Django 5.x) como función de derivación de clave. Las contraseñas nunca se almacenan en texto claro; el campo `password` del `DjangoUser` interno persiste el hash en formato `pbkdf2_sha256$260000$<salt>$<hash>`. Se configura un validador de complejidad personalizado (`ComplejidadPasswordValidator`) que requiere mínimo 8 caracteres, al menos una mayúscula, una minúscula, un dígito y un símbolo especial.

12.2 Tokens de Recuperación de Contraseña

Los tokens se generan mediante `secrets.token_urlsafe(32)`, produciendo 43 caracteres Base64-URL-safe con alta entropía criptográfica. Cada token tiene una ventana de validez de exactamente 1 hora desde su creación, un uso único garantizado mediante el flag `usado=True` tras el reset exitoso, y auditoría de `ip_solicitud` para trazabilidad.

12.3 Protección CSRF

`CsrfViewMiddleware` está activo en toda la aplicación. La directiva `{% csrf_token %}` se incluye en todos los formularios de plantilla. Los endpoints de reconocimiento facial están marcados con `@csrf_exempt` durante el desarrollo; esta configuración debe reemplazarse por autenticación basada en token antes del despliegue en producción.

12.4 Resumen del Modelo de Auditoría

Mecanismo	Evento registrado	Modelo de persistencia
SesionUsuario	Cada inicio de sesión: IP, user-agent, timestamps de inicio y cierre	SesionUsuario
IntentoReconocimientoFacial	Todos los intentos de login facial con similitud y resultado	IntentoReconocimientoFacial
LogActividad	Cada acción CRUD significativa en cualquier módulo	LogActividad (UUID)
HistorialMaquina	Cada modificación de estado o datos de una máquina	HistorialMaquina
MaquinaEliminada	Backup JSON completo antes de cada eliminación física	MaquinaEliminada
BackupDatabase	Cada ejecución de backup automático o manual	BackupDatabase (UUID)

13. Diagrama Entidad-Relación Resumido

Las relaciones clave entre entidades del sistema se estructuran en torno a dos entidades centrales: Usuario y Maquina. Todas las demás entidades se vinculan a una o ambas mediante Foreign Keys o relaciones Many-to-Many.

13.1 Relaciones desde Usuario

Entidad origen	Tipo de relación	Entidad destino
Maquina	FK (responsable, created_by)	Usuario
MantenimientoProgramado	FK (tecnico_asignado, tecnico_realizado) + M2M (personal_adicional)	Usuario
Documento	FK (creado_por, modificado_por) + M2M (usuarios_acceso)	Usuario
Reporte	FK (usuario_solicitante)	Usuario
Notificacion	FK (usuario)	Usuario
LogActividad	FK (usuario)	Usuario
BackupDatabase	FK (creado_por, restaurado_por)	Usuario
ReconocimientoFacial	OneToOne	Usuario

13.2 Relaciones desde Maquina

Entidad dependiente	Tipo de relación	Comportamiento ON DELETE
AlertaMaquina (1:N)	FK → Maquina	CASCADE
HistorialMaquina (1:N)	FK → Maquina	CASCADE
MantenimientoProgramado (1:N)	FK → Maquina	CASCADE
UsoMaquinaria (1:N)	FK → Maquina	CASCADE

Entidad dependiente	Tipo de relación	Comportamiento ON DELETE
ObjetoMaquinaria (1:N)	FK → Maquina	CASCADE
Documento (N:1, opcional)	FK → Maquina	CASCADE (si aplica)
PiezaInventario (N:1, opcional)	FK → Maquina	SET_NULL
Notificacion (contextual)	FK → Maquina (null)	CASCADE

14. Consideraciones de Despliegue

14.1 Variables de Entorno Requeridas

El sistema utiliza django-environ para cargar configuración sensible desde un archivo .env. Las siguientes variables deben estar correctamente definidas antes del despliegue:

- SECRET_KEY — clave criptográfica de Django; debe ser única, aleatoria y confidencial
- DATABASE_URL — cadena de conexión MySQL (usuario, contraseña, host, puerto, nombre de BD)
- REDIS_URL — URL de conexión al servidor Redis (Celery broker + caché)
- EMAIL_HOST_USER / EMAIL_HOST_PASSWORD — credenciales SMTP de Gmail
- ALLOWED_HOSTS — lista de dominios autorizados (cambiar de ['*'] a dominios específicos)
- DEBUG — establecer explícitamente en False

14.2 Checklist Pre-producción

Ítem	Acción requerida
DEBUG	Establecer en False en settings.py o variable de entorno
ALLOWED_HOSTS	Restringir a dominios y/o IPs específicas del servidor
CSRF en endpoints faciales	Reemplazar @csrf_exempt por autenticación por token
SECRET_KEY	Generar nueva clave con <code>django.core.management.utils.get_random_secret_key()</code>
collectstatic	Ejecutar <code>python manage.py collectstatic</code> antes de iniciar el servidor
Migraciones	Ejecutar <code>python manage.py migrate</code> en la BD de producción
Redis y Celery	Verificar conectividad y que los workers de Celery estén activos
VISION_MODEL_PATH	Verificar que el archivo <code>heavy_machinery.tflite</code> esté en la ruta configurada
Backups	Confirmar que <code>mysqldump</code> esté en el PATH del sistema operativo del servidor

15. Glosario Técnico

Término	Definición
ORM	Object-Relational Mapper: capa de abstracción que mapea modelos Python a tablas de BD
MTV	Model-Template-View: patrón arquitectónico de Django (equivalente funcional a MVC)
PBKDF2	Password-Based Key Derivation Function 2: algoritmo de hashing de contraseñas con sal y múltiples iteraciones
Embedding vectorial	Representación numérica de alta dimensión de datos complejos (en este caso, un rostro en 128 dimensiones)
Similitud coseno	Métrica que mide el ángulo entre dos vectores; rango [0, 1], donde 1 indica identidad perfecta
TFLite	TensorFlow Lite: variante optimizada de TensorFlow para inferencia en dispositivos de baja potencia o CPU
Celery	Framework de colas de tareas distribuidas para Python; permite procesamiento asíncrono
APScheduler	Advanced Python Scheduler: librería para ejecución de tareas programadas en procesos Python
DRF	Django REST Framework: librería para construcción de APIs REST sobre Django
FAR / FRR	False Acceptance Rate / False Rejection Rate: métricas de evaluación de sistemas biométricos
UUID	Universally Unique Identifier: identificador de 128 bits usado como clave primaria en modelos críticos
CSRF	Cross-Site Request Forgery: ataque que explota la confianza del servidor en el navegador del usuario
Rate limiting	Control del número máximo de solicitudes permitidas desde una IP en un intervalo de tiempo
Checksum SHA-256	Huella criptográfica de 256 bits usada para verificar la integridad de archivos almacenados